

investigation_20260925

BOB-238 investigation — verdicts for BOB-088 / BOB-106 / BOB-110 / BOB-159 / BOB-162 against commit 7b45113

Investigation date: 2026-09-25 **Investigated commit:**

7b45113c6b03524d9c799bdd68496ec202575e4c

(2026-08-21T21:56:22+02:00, “fix(002,T041): close the review’s 5
IMPORTANT + 7 MINOR findings, and three defects nobody had
reported”) **Method:** git show

7b45113c6b03524d9c799bdd68496ec202575e4c --stat for the touched-file
list, then git show 7b45113c6b03524d9c799bdd68496ec202575e4c --
<file> for each item’s own scope, cross-referenced against each item’s
current full text in docs/Issues.md / docs/Fixed.md. No source, test,
config, or DB file was modified during this investigation — pure read-
only git show/grep/sed against the tree.

Scope note: BOB-107 and BOB-114 were already confirmed case (a) by
the conductor before this dispatch (cited in BOB-238’s own body) — not
re-investigated here. This report covers the 5 remaining UNVERIFIED
ids.

BOB-088 — “RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness”

Verdict: (a) — the commit already closes it.

Item's stated acceptance (from docs/Issues.md): "Complete/verify the README Tracked-Items + Status Documents table row-completeness (GA-07 remaining half). Not independently re-verified this round whether every mandated doc (CONTINUATION.md, Issues.md, Fixed.md, PORTING-FROM-LAVA.md, both new GA/RD2 audit docs, every Status.md/Status_Summary.md pair) actually has a row."

Citing evidence:

1. `git show 7b45113 --stat` shows a brand-new, executable generator script `scripts/testing/update_readme_doc_links.sh` (259 lines, new file mode 100755), plus its §11.4.18 companion doc `docs/scripts/update_readme_doc_links.md` (114 lines, new) and a regression test `tests/unit/test_update_readme_doc_links_no_duplication.sh` (133 lines, new).
2. The generator script's own in-source header states verbatim (diff of `scripts/testing/update_readme_doc_links.sh`):

```
> # tracked file  
itself in THIS invocation; a table cell is never >  
# hand-typed or assumed correct because it "looked right last  
time" > # (§11.4.6). > and its cross-reference block: > # Cross-  
references: docs/scripts/update_readme_doc_links.md (§11.4.18 >  
# companion), CLAUDE.md / constitution/CLAUDE.md §11.4.57  
(the mandate > # this script implements), BOB-088 (the  
tracked item this script closes).
```
3. The companion doc `docs/scripts/update_readme_doc_links.md` (new file, this commit) states the exact measured drift BOB-088's acceptance text asks about, and names its own closing effect verbatim:

```
> "Before this script existed, that table was hand-  
maintained, and it had drifted: measured > 2026-08-21 (BOB-088),  
10 of its 18 rows carried a stale Revision/Last modified pair ... two >  
rows read – (not yet exported) for HTML/PDF cells that had real  
exports on disk, and one > MANDATED document (docs/  
GOVERNANCE_AUDIT_2026-08-08_ROUND2.md, cited by name in  
BOB-088's own > body text) had no row at all." > ... "Cross-  
references: ... BOB-088 — the tracked workable item this script  
closes."
```
4. The diff of `README.md` itself is direct evidence the table was actually regenerated and completed in this same commit — not just scripted for later use:
 - PORTING-FROM-LAVA row: HTML/PDF cells flip from – (not yet exported) to real links (`[HTML] (docs/PORTING-FROM-LAVA.html) / [PDF] (docs/PORTING-FROM-LAVA.pdf)`).

- GOVERNANCE_AUDIT_2026-08-07 row: HTML/PDF cells flip from – (not yet exported) to real links.
- GOVERNANCE_AUDIT_2026-08-08_ROUND2 — the item literally named as missing in BOB-088’s own acceptance text (“both new GA/RD2 audit docs”) — appears as a brand-new row for the first time: | GOVERNANCE_AUDIT_2026-08-08_ROUND2 | 2026-08-09T12:54:30Z | 6 | ... |.
- Issues row’s Revision cell corrects from stale 10 to live 40; Fixed corrects from 20 to 25; every Status.md/Status_Summary.md pair present in the pre-commit table is present and current post-commit.
- The table is now bounded by machine-generated markers (<!-- doc-link-section:begin -->/<!-- doc-link-section:end -->) with a note directly above it reading “Regenerated by scripts/testing/update_readme_doc_links.sh — never hand-typed (§11.4.57).”

5. Anti-bluff proof cited in the same companion doc: “tests/unit/test_update_readme_doc_links_no_duplication.sh ... run against the pre-fix script and confirmed to fail 3/5 assertions before the fix, then pass 5/5 after” — a real RED→GREEN pair, not a claimed-only fix.

Assumption: BOB-088’s acceptance text does not literally use the word “close”; it asks to “complete/verify” the table. I read the generator’s own explicit self-citation (“the tracked item this script closes”) plus the concrete before/after diff evidence (every explicitly-named gap in BOB-088’s body text is filled) as jointly satisfying that acceptance. The commit message’s separate claim of an “orphan sweep [that] went 96 orphans of 305 tracked docs -> 0/305” could not be independently located as a discrete artifact in this diff (no orphan count is printed by update_readme_doc_links.sh itself, whose own doc describes its scope as deliberately narrower than the full §11.4.212 reachability obligation) — I treat that specific “96→0/305” figure as UNCONFIRMED: on its own, but it does not change BOB-088’s own verdict, since BOB-088’s acceptance text is about the Tracked-Items table specifically, and that is fully, concretely closed by the evidence above.

BOB-106 — “§11.4.238 followup: §11.4.84 quiescence-check helper for the unattributed auto-commit path”

Verdict: (b) — the commit partially addresses it.

Item’s stated acceptance (from docs/Issues.md): “Author a §11.4.84 quiescence-check helper (e.g. challenges/scripts/no_unattributed_autocommit_challenge.sh) that scans the commit range since the last known-good release tag and FAILs on any commit message matching a closed bare/templated pattern ... **wired into scripts/pre_build_verification.sh or the §11.4.234 commit-push-all.sh entrypoint.**”

What landed (real progress):

1. New executable guard scripts/hooks/unattributed-commit-guard.sh (237 lines, new file), which implements exactly the described check: closed bare/templated pattern set (^Auto-commit\$, ^sync:[[:space:]]), structure-not-substring matching (§11.4.201(7) (a)), a ticket/task-PR reference exemption, and a --self-test mode implementing §11.4.107(10) golden-good/golden-bad/false-positive self-validation.
2. Companion doc docs/scripts/unattributed-commit-guard.md (new, 82 lines) and hermetic executing test tests/hooks/test_unattributed_commit_guard.sh (new, 140 lines) driving the REAL invocation path (not a bash -n parse check), with a documented paired §1.1 mutation.
3. Real-history proof: run against this repo’s actual git log, the guard names the 14 unattributed commits since the last tag exactly as the commit message claims (“BOB-106/107: two guards built. The commit guard names 14 unattributed commits since the last tag”).

What is missing (explicit, unmet acceptance clause — the wiring):

The guard’s own in-source header states this verbatim, as a section literally titled “WIRING (honest gap, §11.4.6)” (diff of scripts/hooks/unattributed-commit-guard.sh):

```
# — WIRING (honest gap, §11.4.6)
# This script is NOT
yet invoked from scripts/pre_build_verification.sh # or
scripts/commit-push-all.sh (the two entry points the tracked
item
```

```
# names as candidates) – both are outside this authoring
pass's granted # edit scope. Wiring it into one of those two
seams remains OWED as the # item's closing step; see the
BOB-106 tracker entry.
```

Cross-checked mechanically: `scripts/pre_build_verification.sh` IS touched by this commit (164 lines changed per `git show --stat`), but `grep -n "unattributed-commit-guard.sh"` against the full commit diff finds the string ONLY inside: the guard script itself, its `docs/scripts/*.md` companion doc, `tests/hooks/test_unattributed_commit_guard.sh` (a test that invokes the guard directly, not a pre-build/commit seam), and — critically — inside the newly-added `docs/Issues.md` text for the **sibling item BOB-162** (filed in this same commit; see below), which independently states: “Neither is invoked by `scripts/pre_build_verification.sh` OR `scripts/commit-push-all.sh`, so neither exerts standing detection pressure (11.4.226: a guard with no execution seam is not coverage; registration is not coverage).” `scripts/commit-push-all.sh` is not in this commit’s touched-file list at all.

Precisely what is still missing to fully close BOB-106: 1. Wiring `unattributed-commit-guard.sh` into `scripts/pre_build_verification.sh` OR `scripts/commit-push-all.sh` as a real invocation seam (currently invoked by nothing but its own test/self-test). 2. An operator decision on the §11.4.224(E) brownfield-adoption question the guard’s own header and BOB-162 raise: wiring the guard AS-IS against the 14 pre-existing unattributed commits would refuse every future commit immediately, so the acceptance criteria explicitly requires either the wiring OR “a registered deferral pointing at” the tracking item (BOB-162) recording that adoption choice — and BOB-162’s own text states plainly “this item is not closed until that answer is recorded,” i.e. as of the commit, that answer had not yet been recorded.

This partial-progress gap is not something this investigation should silently complete: per the task’s own scope boundary, wiring a guard into `pre_build_verification.sh/commit-push-all.sh` when its own in-source comment explicitly defers that decision to the operator (§11.4.66/§11.4.224(E) brownfield adoption) is exactly the kind of decision reserved for the operator, not a low-risk mechanical follow-up a subagent should make unilaterally. I did not attempt it.

BOB-110 — “BOB-074 followup: UX-class test coverage (accessibility/usability) absent”

Verdict: (a) — the commit already closes it.

Item’s stated acceptance (from docs/Issues.md): “docs/testing/test_type_matrix.md’s §11.4.27 test-type audit found UI functional coverage (Vitest + Playwright) but nothing framed around usability/accessibility/UX outcomes specifically. Scope an axe-core or equivalent accessibility pass over the Angular frontend, covering WCAG checks, keyboard-nav coverage, and screen-reader labeling.”

Citing evidence:

1. `git show 7b45113 --stat` shows a brand-new tests/ux/ suite: `conftest.py` (137 lines), `test_accessibility_axe.py` (141 lines), `test_keyboard_navigation.py` (111 lines), `test_live_dashboard_accessibility.py` (72 lines), 4 fixture HTML files (good/bad pairs for accessibility and keyboard-nav), plus docs/testing/ux_accessibility.md (187 lines, new documentation) and frontend/package.json/package-lock.json changes adding axe-core as a real npm devDependency (not vendored by hand).
2. docs/testing/ux_accessibility.md (new this commit) explicitly maps its coverage table against the 3 named acceptance criteria:
 - **WCAG checks** — real axe-core scan of a real Playwright-rendered, JS-hydrated DOM: image alt-text (`image-alt`), form-control labels (`label`), colour contrast (`color-contrast`), single `<h1>/heading order` (`heading-order`), `<html lang>` (`html-has-lang`).
 - **Keyboard-nav coverage** — real `page.keyboard.press("Tab")` traversal asserting on `document.activeElement.id`, plus real computed `:focus-visible` style checks (`outlineStyle/outlineWidth/boxShadow`) read after a real Tab press.
 - **Screen-reader labeling** — covered via axe’s accessible-name/ARIA-resolution rule set (the doc’s own oracle-selection table states the chosen oracle “Implements the actual WCAG 2.x success criteria (real computed contrast, real ARIA resolution, real accessible-name computation)”), plus the `label` and `image-alt` rules directly assert screen-reader-relevant labeling.

3. Anti-bluff RED→GREEN evidence pasted in the doc:
ally_good.html's alt attribute was removed, the golden-good suite observed to FAIL on image-alt, the file restored with a verified byte-identical sha256sum match, then re-observed to PASS.
4. Golden-FALSE (negative control) cases are documented and asserted: a decorative alt="" image and an aria-hidden="true" icon inside an aria-label-carrying button are both proven NOT flagged, proving the oracle discriminates rather than blanket-failing the fixture.
5. The suite found a REAL defect on the first live run (21 WCAG AA colour-contrast violations on the dashboard) and, per §11.4.238 (automated QA must be the discoverer), left the assertion honestly FAILING rather than silencing it — this is the correct, constitution-mandated behaviour for newly discovered coverage, not a sign BOB-110 itself is incomplete: the item's acceptance text only asks to *scope* the accessibility pass, not to guarantee the underlying app has zero accessibility defects.
6. Confirming that the discovered defect was NOT lost and was NOT conflated with BOB-110's own scope: a separate tracked item, BOB-164 ("Live dashboard fails WCAG AA colour contrast on 21 nodes — brand heading measures 1.43:1 against a 3:1 floor"), exists in docs/Fixed.md with
****Created-By:**** BOB-110 UX-class coverage, discovered by the new axe-core suite on its first live run — and BOB-164 has since been independently CLOSED (it lives in Fixed.md, not Issues.md). This confirms the discovery→tracked-item→fix pipeline worked exactly as the constitution mandates, and that the found defect is fully accounted for outside BOB-110's own scope.

Assumption: BOB-110's acceptance text does not explicitly require covering every route/dialog in the Angular app; the doc itself lists honest gaps (only the dashboard landing view scanned live; the ng serve-hosted route set not scanned directly — a documented, reasoned scope narrowing, not a silent omission). I read BOB-110's acceptance as "scope [build] an accessibility pass covering these three classes of check" (which was done, comprehensively, with real oracle proof), not as "achieve zero accessibility violations across the whole app" (which is explicitly BOB-164's separate, already-closed scope). Under that reading the verdict is (a).

BOB-159 — “Warm ./start.sh over an already-running stack leaves the FR-004d repair window open”

Verdict: (c) — the commit’s mention is unrelated/contextual (specifically: the commit FILES this item to track work it explicitly did NOT do).

Citing evidence — exact context of the mention in the commit message:

“Warm path still has the window; bounded (needs a stale marker AND a live stack together) and filed as BOB-159. The call-site comment that claimed the gate runs”before any container writes” was a misstatement in our own source and now states the real boundary.”

and, at the very bottom of the commit message, under a section literally listing new items:

“Filed this round: BOB-159, BOB-160, BOB-161, BOB-162, BOB-163.”

Mechanical confirmation this is item *creation*, not closure: `git show 7b45113 -- docs/Issues.md` shows the `## BOB-159 – Warm ./start.sh over an already-running stack leaves the FR-004d repair window open` heading appearing as a brand-new added (+) block in the diff — the item did not exist in `docs/Issues.md` before this commit. It is therefore structurally impossible for this commit to have “closed” an item it simultaneously created.

Confirming the commit explicitly declined to do BOB-159’s own scope of work, from the item’s own “prior item text” (i.e. the text as originally filed by this commit, still preserved verbatim in the live `docs/Issues.md` entry today, below a later 2026-08-26 operator-decision addendum):

“The `--recreate` path was fixed this round:
`run_ownership_precondition -> stack_down ->`
`run_ownership_repair -> stack_up ...` **The WARM path is NOT covered by that fix and this item tracks the remainder. ... NOT DONE THIS ROUND, with the reason stated rather than**

hidden: the clean guard is a cheap marker-only probe mode on `scripts/ownership_repair.sh ...` Deferred to this item rather than done unilaterally or silently.”

BOB-159’s own “Affected scope / file-scope manifest” line (`start.sh` (warm-start dispatch, `run_ownership_gate` call site); `scripts/ownership_repair.sh` (marker fast-path)) names exactly the work the commit says it did NOT do. The commit’s actual substantive fix (–recreate ordering: `run_ownership_precondition` → `stack_down` → `run_ownership_repair` → `stack_up`) is a *different, already-completed* scope (IMPORTANT-2 from the T041 review), explicitly distinguished in the commit message from the still-open warm-path gap that BOB-159 tracks.

Status confirmation: `docs/Issues.md` currently shows BOB-159 as ****Status:**** Queued, with a 2026-08-26 OPERATOR DECISION recording HOW the warm-path fix should eventually be implemented (repair, then re-verify until stable) — a decision made 5 days *after* commit 7b45113, further confirming the fix itself has still not landed as of this investigation.

No follow-up work attempted: BOB-159’s acceptance criteria (a bounded, honestly-reporting repair loop on the warm-start path, asserted behaviourally with paired §1.1 mutations) is real implementation work against a now-operator-decided design, not a small disjoint mechanical fix — outside this task’s low-risk-optional-work exception.

BOB-162 — “Two new guards exist but no seam invokes them — plus the brownfield adoption decision the commit guard needs”

Verdict: (c) — the commit’s mention is unrelated/contextual (specifically: the commit FILES this item, in the same breath explicitly stating it is NOT closed).

Citing evidence — exact context of the mention in the commit message: BOB-162 is not named inline in the commit message’s prose sections at all; it appears ONLY in the closing roll-call line:

“Filed this round: BOB-159, BOB-160, BOB-161, BOB-162, BOB-163.”

Mechanical confirmation this is item *creation*, not closure: `git show 7b45113 -- docs/Issues.md` shows the `## BOB-162 – Two new guards exist but no seam invokes them – plus the brownfield adoption decision the commit guard needs heading as a brand-new added (+) block` — the item did not exist before this commit.

****Reported-Via:**** §11.4.202 reporting directive `\task` on 2026-08-21T19:35:02Z`` (a timestamp ~21 minutes before the commit's own 19:56:22Z UTC authoring time) confirms it was filed as a live §11.4.202 reporting-directive action during the same working session that produced this commit, then included in the commit's docs sync.

The item's own body text, written and committed in the SAME commit, explicitly states it is unresolved (quoted from the `docs/Issues.md` diff added by this commit):

“But the GUARDS themselves are still invoked by nothing.” ...
“Neither is invoked by `scripts/pre_build_verification.sh` or `scripts/commit-push-all.sh`, so neither exerts standing detection pressure (11.4.226: a guard with no execution seam is not coverage; registration is not coverage).” ... “But it is the operator's call **and this item is not closed until that answer is recorded.**”

Acceptance criteria (from the same, self-authored text): “Each guard is invoked by a named seam, OR carries a registered deferral pointing at this item. For `unattributed-commit-guard.sh` specifically, the operator has answered the adoption question below and the answer is recorded as consumer DATA — never an invented ratchet.” As of this investigation (`docs/Issues.md`, live), BOB-162 remains ****Status:**** Queued and carries no recorded operator answer to the (a)/(b)/(c)/(d) brownfield-adoption menu it poses — confirming the acceptance criteria genuinely remains unmet, not merely undocumented.

No follow-up work attempted: BOB-162's own acceptance criteria requires either (1) wiring the guards into a real seam — an operator-owned decision per its own explicit text, same reasoning as BOB-106 above — or (2) an operator's recorded answer to a 4-option policy question (11.4.224(E) brownfield adoption). Neither is a small, safe, disjoint mechanical fix a subagent should make unilaterally; both are explicitly reserved for the operator by the item's own text.

Summary table

Item	Verdict	One-line reason
BOB-088	(a) closes it	New update_readme_doc_links.sh generator + regenerated README table; script's own header + companion doc explicitly say "BOB-088 — the tracked workable item this script closes"; every named-missing row now present.
BOB-106	(b) partial	Guard unattributed- commit-guard.sh authored + tested + self-validated, but its own header admits "NOT yet invoked from scripts/ pre_build_verification.sh or scripts/commit-push-all.sh ... Wiring ... remains OWED" — the explicit acceptance-criteria clause.
BOB-110	(a) closes it	Full tests/ux/ axe-core + keyboard-nav + screen- reader-labeling suite built with real oracle, RED→GREEN proof, negative controls; discovered defect correctly spun into its own (now separately Fixed) item BOB-164, not silenced.
BOB-159	(c) unrelated/ contextual (item- filing)	Commit explicitly created this item ("filed as BOB-159") to track warm- path work it says plainly it did NOT do this round; still Queued today.
BOB-162		Commit created this item via a §11.4.202 report

Item	Verdict	One-line reason
	(c) unrelated/ contextual (item- filing)	during the same session, whose own text states “this item is not closed until that answer is recorded”; still Queued today with no recorded operator answer.